

EE272 Homework 3

CNN Accelerator - Part 2

Due: Jan 29, 2026, 11:59 pm

Released: Jan 22, 2026

1 Overview

Now that you have designed all the blocks of a CNN accelerator, you will connect them together to create the top level accelerator module and test it on real convolutional layers. The starter code is at <https://code.stanford.edu/ee272/dnn-accelerator-top-rtl.git>. To get started, log in to the FarmShare machine and run Singularity image as before, and then clone the homework repository:

```
$ git clone https://code.stanford.edu/ee272/dnn-accelerator-top-rtl.git
```

Make sure you are at the top level of this code repository before running any commands:

```
$ cd dnn-accelerator-top-rtl
```

2 CNN Accelerator Interface

The interface of your accelerator is reproduced below. The interface has a clock (`clk`), an active low reset (`rst_n`), and four sets of signals, each for inputs, weights, outputs and layer parameters/configuration. You should ignore the parameter values shown in the interface—these are simply some default values. The true values of these parameters that we will be using are in the top-level testbench. Each of these sets uses a ready-valid protocol. We have also included the figure from homework 2 in Figure 1, which should serve as a visual reference for how the blocks are connected at the top level.

```

module conv
#(
    parameter IFMAP_WIDTH = 16,
    parameter WEIGHT_WIDTH = 16,
    parameter OFMAP_WIDTH = 32,

    parameter ARRAY_WIDTH = 4,
    parameter ARRAY_HEIGHT = 4,

    parameter WEIGHT_BANK_ADDR_WIDTH = 8,
    parameter WEIGHT_BANK_DEPTH = 256,
    parameter IFMAP_BANK_ADDR_WIDTH = 8,
    parameter IFMAP_BANK_DEPTH = 256,
    parameter OFMAP_BANK_ADDR_WIDTH = 8,
    parameter OFMAP_BANK_DEPTH = 256,

    parameter CONFIG_DATA_WIDTH = 8,
    parameter CONFIG_ADDR_WIDTH = 8,

    parameter WEIGHT_FIFO_WORDS = 1,
    parameter IFMAP_FIFO_WORDS = 1
)
(
    input clk,
    input rst_n,

    input [IFMAP_FIFO_WORDS*IFMAP_WIDTH - 1 : 0] ifmap_data,
    output ifmap_rdy,
    input ifmap_vld,

    input [WEIGHT_FIFO_WORDS*WEIGHT_WIDTH - 1 : 0] weight_data,
    output reg weight_rdy,
    input weight_vld,

    output [OFMAP_WIDTH - 1 : 0] ofmap_data,
    input ofmap_rdy,
    output reg ofmap_vld,

    input [CONFIG_ADDR_WIDTH + CONFIG_DATA_WIDTH - 1 : 0] config_data,
    output reg config_rdy,
    input config_vld
);
// Implementation
endmodule

```

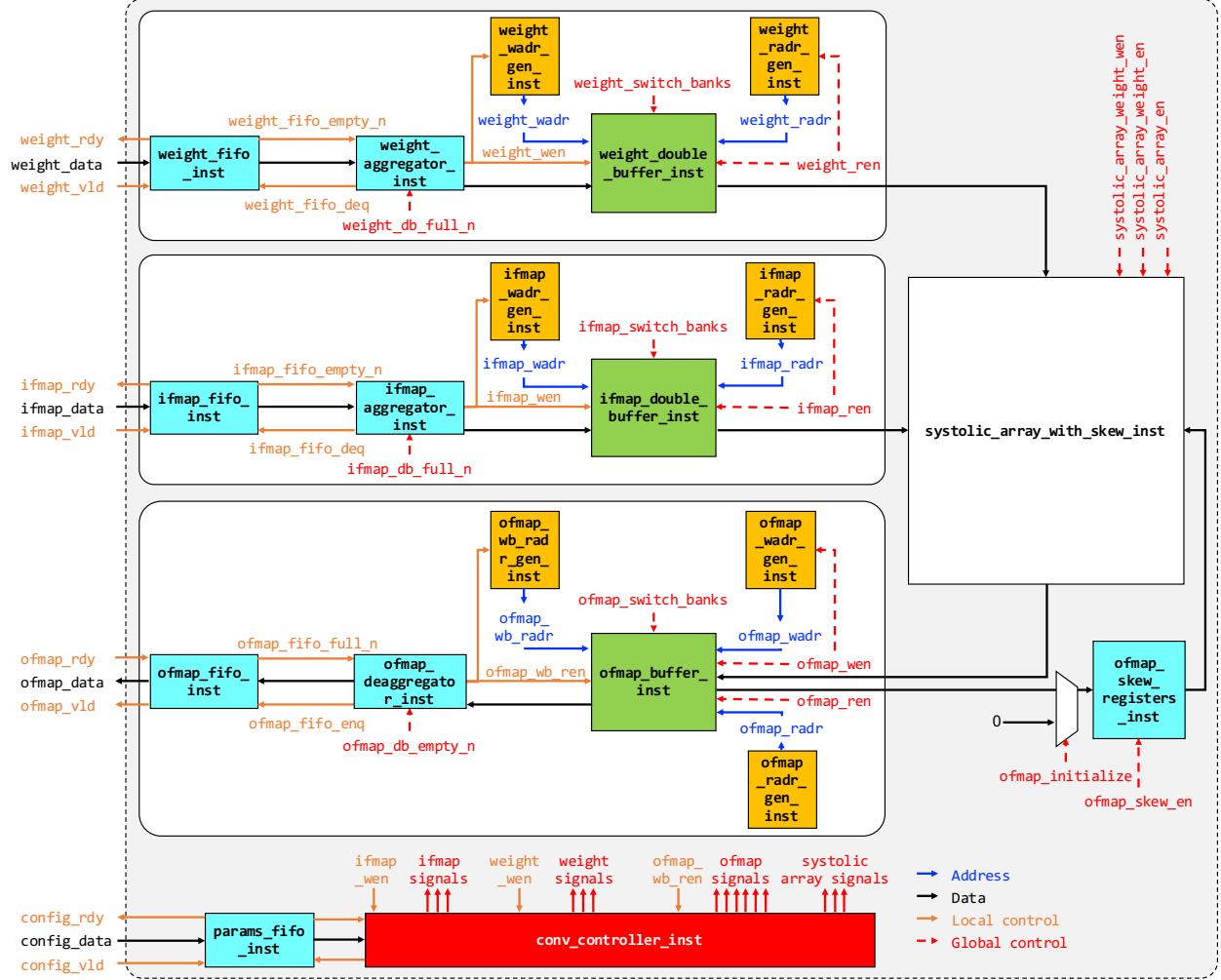


Figure 1: Modules in the CNN accelerator.

2.1 Data Interface

As discussed in class, the number of input/output (I/O) ports going into an accelerator are typically limited, so I/O bandwidth is always a constraint. We have ready/valid ports for inputs, weights and outputs, and **the ports have a smaller width than the width of the corresponding double buffer**. In other words, the inputs, weights and outputs are serialized to some extent to keep the number of I/O ports of the accelerator reasonable.

The input ready/valid port (`ifmap_data`, `ifmap_rdy`, `ifmap_vld`) is connected to an input FIFO (`ifmap_fifo_inst`). The code snippet below shows this connection:

```

fifo
#(
    .DATA_WIDTH(IFMAP_FIFO_WORDS*IFMAP_WIDTH),
    .FIFO_DEPTH(3),
    .COUNTER_WIDTH(1)
) ifmap_fifo_inst (
    .clk(clk),
    .rst_n(rst_n),
    .din(ifmap_data),
    .enq(ifmap_rdy_w && ifmap_vld),
    .full_n(ifmap_rdy_w),
    .dout(ifmap_fifo_dout),
    .deq(ifmap_fifo_deq),
    .empty_n(ifmap_fifo_empty_n),
    .clr(1'b0)
);

assign ifmap_rdy = ifmap_rdy_w;

```

The FIFO takes in `IFMAP_FIFO_WORDS` pixels at a time, each pixel being `IFMAP_WIDTH` bits wide. The not full (`full_n`) signal is connected to the `ifmap_rdy` port—this indicates that the input port is ready to accept data. The `enq` signal of the FIFO is connected to `ifmap_rdy_w && ifmap_vld`, meaning that you should enqueue into this FIFO only when it is ready (there is space in it) and the data you are receiving is valid. You can similarly analyze the weight and output ready/valid ports and their connections to the corresponding FIFOs.

2.2 Configuration Interface

In addition to the data FIFOs, we also need a FIFO to feed in the tiling parameters for the layer we want to run on the accelerator. There are a lot of parameters, so it is important to send them in one by one, so we don't use a lot of I/O ports. These parameters include values for `OX0`, `OX1`, etc., the max addresses of the weight, ifmap, and ofmap buffers, and useful loop bounds like the product of `IC1`, `FY`, `FX`, `OY0`, and `OX0`. We need this loop bound in `conv_controller` to generate the enables that get sent to the rest of the accelerator modules. We wouldn't want to do this multiplication in hardware (it is a constant for the duration of the convolution), so it is more efficient to pass it in as a parameter.

All of these parameters are sent into the accelerator in chunks of 2 bytes. The first byte is an address, and the second byte is the value of the parameter. The parameter FIFO passes these into the convolution controller, which writes them to the appropriate configuration registers. We have arbitrarily assigned addresses to the different configuration (parameter) registers in the controller. The parameters that need more than 8 bits are sent over multiple cycles in chunks of 8 bits. A special address (the last one) tells the `conv_controller` that all the parameters have been sent in, and it can now start the execution of the layer.

3 Remaining Modules

3.1 Aggregator

The weight/ifmap aggregator reads weight/ifmap pixels from the interface FIFO. It collects several pixels and then writes them as one wide word into the double buffer. For example, before writing into the ifmap double buffer, we need to collect IC0 pixels and write them all at once. Similarly, before writing into the weight double buffer, we need to collect OC0 weights.

3.2 Deaggregator

The deaggregator performs the inverse of the aggregator. The deaggregator reads several (OC0) ofmap pixels in parallel from the accumulation buffer, unpacks them, and sends them out to the ofmap interface FIFO serially (one pixel at a time).

3.3 Convolution Controller

Convolution controller is the state machine that is responsible for generating control signals that orchestrate the flow of data through the accelerator. These control signals include all of the red signals in Figure 1.

The convolution controller is the most complex block in the accelerator, and requires a few weeks to implement correctly from scratch. Since we are time-constrained, we are providing you with a complete implementation of this module. However, understanding how it works is important, especially because that will assist you in writing the C++ version of the accelerator that we will push through high-level synthesis in the next homework.

Please read the implementation of convolution controller in `verilog/conv_controller.v` and answer these questions (they will be reproduced in the report). **The comments in this code should give you most of the information you need to understand how the controller is working.** You should include an annotated screenshot of a waveform in the answers to receive full credit. The annotations should be in terms of configuration parameters (tiling, filter size, etc.) rather than absolute numbers. Keep your answers to 2-3 sentences.

1. When is the weight double buffer read enabled (`weight_ren`)? Why is it enabled at these times?
2. When is the input double buffer read enabled (`ifmap_ren`)? Why is it enabled at these times?
3. When is the systolic array enabled (`systolic_array_en`)? Why do we wait to enable it?
4. At what state of the finite state machine do we switch the input buffer banks? Why don't we switch it at the same time as the weight and output buffers?
5. Why is the product of IC1, FY, FX, OY0 and OX0 sent as a parameter into the convolution controller? How is it used in determining when `systolic_array_en` is high?

3.4 Convolution Top Level Module

Finally, you will connect all the blocks you have designed to create the top level module of the accelerator in `verilog/conv.v`. Please read the comments in the file carefully. We have already created all the wires you need and the comments in the code describe the connections in detail. Make sure to instantiate and connect:

1. Weight double buffer and address generators
2. Input double buffer and address generators
3. Systolic array with skew registers
4. Accumulation buffer
5. Output skew registers
6. Accumulation buffer address generators

Refer to Figure 1 to make sure you have connected everything correctly.

4 Integration Testing

To test your top level convolution module, we have provided a testbench called `conv_tb.v` which feeds inputs and weights into the accelerator in the appropriate order and checks the outputs from the accelerator against the output of the gold model. This testbench is parameterized, it takes in a file called `tests/layer_params.v`. Similarly, the gold model takes in a file called `cpp/conv_tb_params.h`. In order for `conv_tb.v` to pass, the parameters in each of these files need to match. We have provided a mechanism for you to write the layer parameters in a JSON file, and then `json_to_parameters.py/autograder.py` rewrites both `tests/layer_params.v` and `cpp/conv_tb_params.h` with the same values from the JSON file.

4.1 Running Tests Manually

To run a layer manually (without the autograder) do:

```
$ python3 json_to_parameters.py -l layers/resnet_conv2_x_params.json
$ make generate_layer
$ make compile_conv
$ make run_conv
```

If you want to debug the results in a waveform viewer, do:

```
$ make debug_conv
```

To clean up generated files, do:

```
$ make clean
```

We recommend doing most of the debugging with a layer or two, perhaps one which has a stride of 1 and another which has a stride of 2. **You should also feel free to create smaller layers (by writing your own json files) that will allow you to debug quickly.** For example, you can use the tiny layer from homework 2 figures. You can also reduce the size of the array to 4×4 but make sure you reduce IC0 and OC0 appropriately.

4.2 Running Tests with Autograder

The autograder does all of the above steps together. Use the following command to test on a layer:

```
$ python3 autograder.py conv_tb -l layers/resnet_conv2_x_params.json
```

If you don't pass in any layer json file, the autograder will use the second ResNet-18 layer (layers/resnet_conv2_x_params.json). You can pass in as many layers at a time as you want, just leave a space in between the JSON file names. Also, if you would like to test all layers, use the command:

```
$ python3 autograder.py conv_tb -l all
```

This will take a **long time to run**, so it would be a good idea to run all layers overnight. **If you want to debug the output of the commands that the autograder is running, use the verbose option -v** or run the layer manually as indicated in the previous subsection.

If you need any help with the commands for the autograder use:

```
$ python3 autograder.py --help
```

Make sure you exercise your testbench on all the provided layer jsons.

5 Evaluating Accelerator Performance

Once you have a functional design, the next step is to evaluate its performance. **In your report, please answer the following questions (questions 2 through 7 in a single table) for an accelerator with a 16×16 systolic array:**

1. The memory sizes you used in your accelerator.
2. The tiling you used for each ResNet-18 layer, meaning the values for IC0, OC0, OX0, OY0, FX, FY, IC1, OC1, OX1 and OY1.

3. The number of cycles your accelerator takes to run each ResNet-18 layer. Also report the breakdown between configuration time and execution time.
4. The ideal number of cycles your accelerator should take for each ResNet-18 layer.
5. The ratio of (3) and (4) above, meaning the utilization of your MAC units.
6. For every layer, write what determines its performance—is it compute bound, or input write bandwidth bound, or weight write bandwidth bound, or output read bandwidth bound. You should determine this analytically, and then match it with what you observe in the waveforms.
7. Percentage utilized read and write bandwidth of each buffer, that is, if your total runtime for a layer is 10 cycles, and of those you read your input double buffer for 3 cycles, then your utilized read bandwidth for input double buffer is $3 / 10$.
8. Suggest concrete ways for improving the performance of the accelerator while keeping the number of MAC units the same (16×16), and without drastically increasing the I/O bandwidth. (For example, you may decide to keep the total number of I/Os the same but redistribute their allocation between inputs, weights, outputs, and configuration.) Suggest concrete ways for reducing the amount of SRAM used without sacrificing performance. **If you implement these improvements you will receive extra credit. In that case turn in both your baseline accelerator and the improved version, and clearly document the difference between the two and quantify the improvement in the report.**

6 Even More Extra Credit!

If you get done with this homework early and would like to help us improve the homework even further for future years and are up for a challenge, here are some ideas to implement that would allow you to earn some more extra credit. (Do not attempt this before you have completely finished the earlier required part of the homework.)

- Currently, we are using a loop mapping that assumes that the entire input channel dimension fits in the input buffer and the weight buffer. So layers with very large input channel dimension (e.g. 512) may necessitate a very large buffer. To solve this, we need to split IC into another tiling level, that is IC0, IC1 and IC2. The IC2 loop will be outside the accelerator, but this change may mean that the accumulation buffer may need to be flushed to DRAM and then read back in depending on the loop ordering you use. This will involve several changes to the controller as well as the test bench.
- We have only tested the accelerator code on square systolic arrays. You could extend the testing to non-square systolic arrays and fix any bugs that appear.
- Currently, the test bench only tests the accelerator on randomly generated data for the input and weight tensors. It would be nice to bring up a test infrastructure that allows running real ResNet-18 (or 50), that is, real weights for the network as well as real

images from ImageNet. This will allow us to measure the accuracy of the hardware. This requires both creating the quantized neural network (remember the lecture on DNN quantization), and adding some scaling factors to the test bench to achieve good accuracy.

7 Code Submission and Grading

Please submit your code as a ZIP file to Gradescope. Also submit a PDF report explaining your strategy and implementation of the accelerator, and answering all the questions we have asked in this document. **Please use the report template, this greatly simplifies the grading process for us. Be concise in your answers.** The grading of this assignment will be based on the correctness of your code and your report. Please name your files using your and your partner's full names:

- `firstname1_lastname1-firstname2_lastname2.zip`
- `firstname1_lastname1-firstname2_lastname2.pdf`